

SGEIA — Evaluation Preparation Guide

What to Say · What to Highlight · How to Score Maximum Marks

Tejeswari S K | Prodapt AFDE Capstone | May 2026

EVALUATION CRITERIA OVERVIEW — SGEIA

Criteria	Weight	Priority
1. Problem Understanding & Domain Alignment	10%	MEDIUM — Know well
2. Data Engineering & Processing	10%	MEDIUM — Know well
3. RAG & Retrieval Quality	15%	HIGH — Prepare deeply
4. Agentic AI Design	15%	HIGH — Prepare deeply
5. Recommendation / Intelligence Quality	15%	HIGH — Prepare deeply
6. System Design & Architecture	10%	MEDIUM — Know well
7. Evaluation Framework	10%	MEDIUM — Know well
8. Dashboard / User Experience	5%	BASIC — Cover clearly
9. Code Quality & Documentation	5%	BASIC — Cover clearly
10. Demo & Presentation	5%	BASIC — Cover clearly

Strategy: The top 3 criteria (RAG, Agentic AI, Recommendation) carry 45% of total marks. Spend the most time preparing those. The next 4 criteria (Problem, Data, System, Evaluation) carry 40%. Last 3 (Dashboard, Code, Demo) carry 15% — be clear and confident but brief.

1. Problem Understanding & Domain Alignment

10%

What the evaluator is looking for:

The evaluator wants to see: you clearly understand WHY this problem exists in the power grid industry, WHO is affected, and WHY AI is the right solution. Show domain knowledge — not just technical implementation.

Exactly what to say (your script):

"Grid operators today face three critical challenges: first, information overload — thousands of sensor readings and alerts arrive every second with no unified view. Second, slow root cause diagnosis — manually correlating SCADA logs, spreadsheets, and maintenance records takes 2 to 8 hours per incident. Third, no conversational interface — existing tools show data but cannot answer WHY questions. SGEIA solves all three. A grid operator simply types: 'Why is Zone A showing instability?' and gets a data-backed, explainable answer in under 5 seconds. This is the exact problem utilities face globally — unplanned outages cost \$18 to \$70 billion annually in the US alone."

Evidence from your SGEIA project:

Business problem: Grid operators spend hours diagnosing faults manually across disconnected systems (SCADA, spreadsheets, maintenance logs). SGEIA provides natural-language intelligence over grid data. Domain relevance: Built specifically for power grid operations — transformers, substations, distribution units, smart meters, voltage/frequency monitoring, Zone A/B/C/D regional analysis. Target users: Grid operators, utility engineers, operations managers — people who need answers fast but may not have data science skills.

Key terms to use in your answer:

- | | | |
|----------------------------|------------------------|------------------------|
| • Power grid operations | • SCADA systems | • Voltage instability |
| • Frequency excursion | • Transformer overload | • Smart meter anomaly |
| • Operational intelligence | • Root cause analysis | • Utility engineers |
| • Grid reliability | • Outage cost | • Real-time monitoring |

Examiner tip:

Open with the business problem, not the technology. Say "Grid operators face X problem" BEFORE saying "we built Y system". Evaluators want to see you understand the domain, not just code.

2. Data Engineering & Processing

10%

What the evaluator is looking for:

Evaluators want to see: how you handled raw datasets, what preprocessing was done, how metadata was extracted and augmented, and why your chunking strategy is appropriate.

Exactly what to say (your script):

"We used two datasets. DS1 is the Smart Grid Stability Dataset from UCI — 60,000 rows of physics-based telemetry with tau, p, and g features representing reaction times, power balance, and price elasticity. DS2 is the Household Power Consumption Dataset — 2 million real minute-by-minute readings including voltage, current, reactive power, and sub-metering per room. Both original datasets lacked operational context, so we augmented them. For DS1, we added: timestamp, region (Zone A/B/C/D), equipment_type, transformer_status, grid_frequency, and outage_event. For DS2, we added region mapping, equipment labels, demand_load, and event tagging. This is essential because real grid operations require knowing WHEN, WHERE, and WHAT EQUIPMENT was involved — not just physics values. For chunking: each of the 200 synthetic incidents becomes one chunk — approximately 40 to 80 words. This is document-level chunking. We chose this because incident descriptions are already self-contained operational narratives. Recursive splitting would break context."

Evidence from your SGEIA project:

DS1: 60,000 rows, UCI Smart Grid Stability Dataset. Features: tau1-4 (reaction time constants), p1-4 (power balance at each node), g1-g4 (price elasticity). Augmented with: timestamp, region, equipment_type, transformer_status, grid_frequency, outage_event. DS2: 2 million rows, Household Power Consumption. Features: voltage, current, power_consumption, reactive_power, sub_metering_kitchen/laundry/hvac. Augmented with: region, equipment_type, outage_event, demand_load, grid_frequency. Incidents CSV: 200 synthetic records generated from DS1 (grid events) and DS2 (meter events). Grid incidents use DS1 physics values; smart meter incidents (meter_dropout, high_demand_event) use real DS2 voltage/current/consumption values. Chunking: Document-level (1 incident = 1 chunk). Why not recursive? Descriptions are 40-80 words — optimal for embedding without splitting operational context.

Key terms to use in your answer:

- Document-level chunking
- Metadata augmentation
- DS1 telemetry
- DS2 smart meter
- tau/p/g features
- outage_event column
- region mapping
- transformer_status
- 60,000 rows
- 2 million rows
- Synthetic incident generation
- Feature engineering

Examiner tip:

Show the data table/columns on screen if possible. Say specifically WHY you added each new column. Evaluators check if you understand the data, not just loaded it.

3. RAG & Retrieval Quality

15%

What the evaluator is looking for:

The highest-weight technical criterion. Evaluators want to see: how your RAG pipeline works end-to-end, why you chose hybrid search, how semantic retrieval works, and how relevant the retrieved information is.

Exactly what to say (your script):

"Our RAG pipeline has 6 stages: load, chunk, embed, index, search, and synthesise. We use HYBRID RETRIEVAL — combining BM25 keyword search and ChromaDB semantic vector search. BM25 catches exact matches: if an operator says Zone_A or transformer or INC-042, BM25 finds it instantly. ChromaDB finds conceptually similar incidents even when words differ. Together, using Reciprocal Rank Fusion, we get 15 to 20 percent better recall than either method alone. For embeddings, we use all-MiniLM-L6-v2 — a 384-dimensional sentence transformer that runs locally on CPU. We chose this because the corporate proxy blocks OpenAI's embedding API. This model gives excellent semantic quality for technical operational text without any internet dependency. The vector store is ChromaDB — local, persistent, supports metadata filtering by region, severity, and equipment type simultaneously. Top 8 results are selected after Cross-Encoder reranking. We also have a query cache — if the same or very similar question was answered before (cosine similarity > 0.97), we return the cached answer instantly without re-running the pipeline."

Evidence from your SGEIA project:

Hybrid RAG: BM25 (rank-bm25 library) + ChromaDB (cosine similarity) → RRF fusion → Cross-Encoder reranking
Embedding model: all-MiniLM-L6-v2, 384-dim, CPU-only, no internet needed
Vector DB: ChromaDB with HNSW graph, persistent on disk, metadata filtering supported
Knowledge base: 200 incidents indexed — hybrid of DS1-sourced grid events and DS2-sourced smart meter events
Query cache: ChromaDB query_cache collection, cosine threshold 0.97 for instant replay
Metadata filters: auto-detected from query — region (Zone_A/B/C/D), severity (critical/high/medium/low), equipment_type (transformer/substation/distribution_unit/smart_meter_bank)
Why not semantic-only: Misses exact zone names and incident IDs. Why not BM25-only: Misses synonyms and concepts.

Key terms to use in your answer:

- Hybrid retrieval
- BM25 keyword search
- Semantic vector search
- RRF fusion
- ChromaDB
- all-MiniLM-L6-v2
- 384-dimensional embeddings
- HNSW graph
- Cross-Encoder reranking

- Cosine similarity
- Metadata filtering
- Query cache
- Reciprocal Rank Fusion
- Top-K retrieval
- Recall improvement

Examiner tip:

Draw or show the retrieval flow diagram: query → BM25 + ChromaDB in parallel → RRF → top 8. Mention specific numbers: 200 incidents, 384 dimensions, cosine > 0.97 for cache. Evaluators love specifics.

4. Agentic AI Design

15%

What the evaluator is looking for:

Evaluators want to see: a clear multi-agent architecture, how agents are orchestrated, how routing decisions are made, and why multiple agents are better than a single LLM call.

Exactly what to say (your script):

"SGEIA uses a 5-agent LangGraph StateGraph. Each agent has a single responsibility and they communicate through a shared AgentState object. Agent 1 — LLM Query Router: reads the user's question and classifies it as incident, stability, smart_meter, cross_domain, or out_of_scope. This is critical — it routes to only the agents needed, making the system faster and more accurate. Agent 2 — Grid Retrieval Agent: executes hybrid BM25 + ChromaDB search and returns the top 8 incidents. Agent 3 — Grid Stability Agent: runs XGBoost classifier on DS1 telemetry, returns health score and SHAP values. Agent 4 — Smart Meter Agent: runs Isolation Forest on DS2 data, returns anomaly flags and consumption stats. Agent 5 — Failure Analysis and Recommendation Agent: synthesises all outputs from agents 2, 3, and 4. Generates root cause explanation and mitigation steps, then validates through DeepEval quality gates. Why LangGraph over single LLM call? A question about incidents needs only agent 2. A stability question needs agents 3 and 5. LangGraph's conditional routing means we never waste time running unnecessary agents. MemorySaver checkpoint also enables multi-turn conversation continuity."

Evidence from your SGEIA project:

Framework: LangGraph StateGraph with MemorySaver checkpoint (conversation memory across turns) 5 Agents: ① LLM Query Router — intent classification (incident/stability/smart_meter/cross_domain/out_of_scope) ② Grid Retrieval Agent — BM25 + ChromaDB hybrid search, RRF fusion, Cross-Encoder reranking ③ Grid Stability Agent — XGBoost on DS1 telemetry → health score + SHAP feature importance ④ Smart Meter Agent — Isolation Forest on DS2 → anomaly score, batch vectorized prediction ⑤ Failure Analysis + Recommendation Agent — synthesis + DeepEval quality gates Routing: Conditional edges in LangGraph graph. Router output determines which agents activate. A2A communication: All agents share AgentState TypedDict. Output of one agent is input to the next. Why not single LLM: Different questions need different data sources. Routing prevents unnecessary computation.

Key terms to use in your answer:

- LangGraph StateGraph
- Multi-agent orchestration
- Conditional routing
- AgentState
- MemorySaver checkpoint
- Agent orchestration
- Intent classification
- A2A communication
- Parallel agent execution
- Modular agent design
- LLM Query Router
- Cross-domain queries

Examiner tip:

Draw the agent flow on a whiteboard or show the diagram. Say: "Agent 1 decides, Agents 2-4 gather data, Agent 5 synthesises." Evaluators check if you understand WHY each agent exists separately.

5. Recommendation / Intelligence Quality

15%

What the evaluator is looking for:

Evaluators want to see: how root cause is identified, how prediction quality is demonstrated, and most importantly — how recommendations are EXPLAINABLE to non-technical operators.

Exactly what to say (your script):

"Root cause identification works through three evidence streams: historical incident retrieval, live telemetry classification, and anomaly detection. For stability, XGBoost classifies the current telemetry as stable or unstable with a probability score. SHAP values then explain WHICH feature caused the prediction — for example, high tau1 means slow node reaction time, which we tell the operator in plain English: 'Zone A nodes are responding too slowly to load changes, which is causing voltage deviation.' For smart meters, Isolation Forest flags which consumption readings are anomalous compared to historical patterns. The Recommendation Agent generates specific mitigation steps grounded in the retrieved incidents. For example: 'Based on 3 similar incidents in Zone A (INC-015, INC-044, INC-142), the recommended action is: 1. Check transformer load limits, 2. Engage automatic frequency regulation, 3. Dispatch crew.' Explainability is built in — every answer shows WHICH incidents were retrieved, WHAT the stability model predicted, and WHY those are the recommended actions. Operators always know the source of every recommendation."

Evidence from your SGEIA project:

XGBoost classifier + regressor on DS1: • Classifier: predicts stable/unstable with probability score • Regressor: predicts continuous stability margin (stab value) • Health score = $(1 - P(\text{unstable})) \times 100$, updated every 60 seconds • SHAP TreeExplainer: identifies which tau/p/g feature drove each prediction Isolation Forest on DS2: • Unsupervised anomaly detection (no labels needed) • Batch vectorized prediction for 2M rows • Returns anomaly score per reading — ranked, not just binary Recommendation quality: • Grounded in retrieved incidents (IDs, descriptions, historical resolutions) • Validated by DeepEval faithfulness check • LLM-as-Judge scores 1-5, threshold ≥ 3 to show to operator • Retry logic: regenerates if score < threshold

Key terms to use in your answer:

- | | | |
|--------------------------|-----------------------|----------------------------|
| • XGBoost classifier | • SHAP values | • Feature importance |
| • Explainable AI | • Root cause analysis | • Isolation Forest |
| • Anomaly score | • Health score 0-100 | • Grounded recommendations |
| • LLM-as-Judge | • Faithfulness check | • Mitigation steps |
| • Prediction probability | • Stability margin | • Evidence-based |

Examiner tip:

Show a LIVE DEMO for this criterion. Type "Why is Zone A unstable?" and show the answer citing actual incident IDs and SHAP values. Evaluators want to see the explanation, not just hear about it.

6. System Design & Architecture

10%

What the evaluator is looking for:

Evaluators want to see: clean microservice design, well-structured API, scalable architecture, and modular code where each component can be changed independently.

Exactly what to say (your script):

"SGEIA is designed as a microservice — a single FastAPI application that serves both the backend API and the frontend dashboard. The architecture follows a clear separation of concerns. The folder structure separates: API layer (src/api/), agent logic (src/agents/), ML models (src/models/), indexing (src/indexing/), guardrails (src/guardrails/), and simulation (src/simulation/). Each folder can be changed independently without affecting others. The API has 8 endpoints: /api/chat/rag for streaming chat, /api/dashboard/metrics for the dashboard, /api/scada/ingest for SCADA integration, /api/feedback for operator ratings, and more. We use Server-Sent Events for streaming — this allows the frontend to show each pipeline step live as it happens. REST would only return the final answer; SSE shows the operator exactly what the AI is doing in real time. The system is scalable: ChromaDB persists to disk, ML models are loaded once as singletons, and the incident cache is loaded into memory at startup so dashboard refreshes take under 100ms."

Evidence from your SGEIA project:

Architecture: FastAPI microservice, serves API + static frontend from one process Folder structure: src/api/, src/agents/, src/models/, src/indexing/, src/guardrails/, src/simulation/ 8 API endpoints: /api/chat/rag (SSE streaming), /api/dashboard/metrics, /api/health, /api/scada/ingest, /api/feedback, /api/incidents, /api/stability/check, /api/telemetry/live Streaming: Server-Sent Events (SSE) — shows pipeline steps live as they execute Scalability features: singleton ML models, in-memory incident cache, ChromaDB disk persistence, LLM fallback chain (Prodapt → Groq → LangChain) Modularity: each src/ subfolder is independent — can swap ChromaDB for another vector DB by only changing chroma_store.py; swap LLM by only changing model_router.py

Key terms to use in your answer:

- FastAPI microservice
- Server-Sent Events
- REST API design
- Modular architecture
- Separation of concerns
- Singleton pattern
- LLM fallback chain
- ChromaDB persistence
- API endpoints
- Uvicorn
- CORS middleware
- Scalable design

Examiner tip:

Show the /docs page (FastAPI auto-generated Swagger UI) — evaluators love seeing a live API spec. Also mention the folder structure briefly to show code organisation.

7. Evaluation Framework

10%

What the evaluator is looking for:

Evaluators want to see: how you validate that AI answers are trustworthy, the DeepEval setup, LLM-as-Judge methodology, and what happens when quality fails.

Exactly what to say (your script):

"Every answer SGEIA generates goes through three independent quality checks before reaching the operator. First: Faithfulness score (0 to 100%). This checks whether the answer actually uses real data from the retrieved incidents. If the LLM makes something up or gives a generic answer, the faithfulness score drops. We check: does the answer cite actual numbers? does it reference zone names? is it longer than 40 words? Second: LLM-as-Judge score (1 to 5). A second AI independently evaluates the answer on four criteria: relevance to the question, specificity of data cited, completeness, and absence of excessive disclaimers. Only answers scoring 3 or above are shown to the operator. Third: Output Safety. We scan the LLM's response for jailbreak patterns, system prompt leaks, and PII before showing it to the user. These three scores are shown in grey below every answer in the chat UI. The operator can see exactly how confident the system is in each answer. If an answer fails quality checks, the system retries once with an improved prompt before showing the result."

Evidence from your SGEIA project:

DeepEval — 3 independent scores per answer: ① Faithfulness (0-100%): Checks answer cites real numbers, incident IDs, zone names from retrieved data. Deductions: -40% if too short, -35% if generic/hallucinated, -15% if no numbers cited. ② LLM-as-Judge (1-5): Independent criteria — question keyword overlap, specificity, word count, disclaimer count. Pass threshold: $\geq 3/5$. Retry once if score < 3 . ③ Output Safety (Pass/Fail): Jailbreak pattern scan, system prompt leak detection, PII re-scan. Scores displayed: shown in grey below every chat answer in the UI. Cache integration: Answers scoring $\geq 50\%$ faithfulness are cached in ChromaDB for instant reuse. Operator feedback loop:   buttons — high-rated answers also trigger cache storage.

Key terms to use in your answer:

- DeepEval
- LLM-as-Judge
- Faithfulness score
- Output safety
- Validation methodology
- Quality gates
- Retry logic
- Hallucination detection
- Grounded answers
- Score threshold
- Operator feedback
- Cache boosting
- Answer quality

Examiner tip:

Show the DeepEval scores in the actual UI — point to the grey text below an answer that shows "Faithfulness: 85% ■ LLM Judge: 4/5 ■ Output Safety: ■". This is live proof of the framework.

8. Dashboard / User Experience

5%

What the evaluator is looking for:

Evaluators want to see: an intuitive interface that a non-technical grid operator can use, operational visibility, and real-time data.

Exactly what to say (your script):

"The SGEIA dashboard has 12 widgets covering every aspect of grid operations: Grid Health Score, Active Incidents, Zone Status Map, Stability Trend, Frequency Monitor, Voltage Distribution, Demand vs Capacity, Equipment Status, Anomaly Feed, Agent Activity, Recommendations, and Incident Breakdown. All data refreshes every 60 seconds from live telemetry. The health score comes from real XGBoost predictions on replayed DS1 data — it genuinely changes every minute. The chat interface is context-aware. A single General Assistance button opens a chat that has full access to all 12 widgets' data. Operators can ask anything in plain English. The splash screen, loading animations, pipeline status bar, and DeepEval scores all give operators real-time visibility into what the system is doing at every step."

Evidence from your SGEIA project:

12 live widgets: Grid Health Score (XGBoost-derived), Active Incidents, Zone Status Map, Stability Trend, Grid Frequency Monitor, Voltage Health Distribution, Demand Load vs Capacity, Equipment Status, Smart Meter Anomaly Feed, Agent Activity Feed, Recent Recommendations, Incident Category Breakdown Chat: General Assistance button → context-aware chat with full dashboard snapshot Real-time: 60-second refresh cycle, live telemetry simulator replays DS1 rows UX features: Splash loading screen, pipeline status bar (shows live steps), DeepEval scores in grey, ■/■ feedback buttons, dark/light/contrast themes

Key terms to use in your answer:

- 12-widget dashboard
- Operational visibility
- Grid Health Score
- Real-time refresh
- Plain English queries
- Zone Status Map
- Context-aware chat
- Live telemetry

Examiner tip:

Open the dashboard in the browser and walk through it live. Click the chat button, type a question, show the pipeline status bar updating in real time. 2-minute demo max.

9. Code Quality & Documentation

5%

What the evaluator is looking for:

Evaluators want to see: clean organised code, readable structure, good README, maintainability.

Exactly what to say (your script):

"The codebase is organised into 7 focused modules under src/: api, agents, models, indexing, guardrails, simulation, and data. Each module has a single responsibility and can be changed independently. Every file has a module-level docstring explaining what it does. Every function has type hints and docstrings. All requests are tagged with a request_id for tracing through logs. The README covers: project setup, environment variables, how to run training scripts, sample queries, and API usage examples. Configuration is centralised in config.py using pydantic-settings — all paths, model names, and API keys are defined once and imported everywhere. No magic strings scattered in code."

Evidence from your SGEIA project:

Project structure: 7 focused src/ modules, each with single responsibility Documentation: Module docstrings, function type hints, request_id on every log line Config: pydantic-settings in config.py — centralised settings, reads from .env automatically Logging: Structured logging with module name + request ID on every log entry README: Setup instructions, training commands, sample queries, API examples Code style: Consistent naming, no magic strings, clean imports, singleton patterns for ML models

Key terms to use in your answer:

- Modular structure
- Single responsibility
- Type hints
- Docstrings
- pydantic-settings
- Structured logging
- README
- request_id tracing

Examiner tip:

Open VS Code and show the folder structure. Point to one agent file and explain its docstring. Show the README. Keep this brief — 1 minute is enough for 5% criteria.

10. Demo & Presentation

5%

What the evaluator is looking for:

Evaluators want to see: confident solution explanation, smooth demo execution, and ability to answer Q&A.;

Exactly what to say (your script):

"Let me show you SGEIA live. [Open browser to <http://127.0.0.1:8000/>] First, the dashboard — 12 widgets showing live grid data, refreshing every 60 seconds from real DS1 telemetry. Now let me show the RAG pipeline in action. [Click General Assistance button, type: 'Why is Zone A showing instability?'] Watch the pipeline status bar — you can see: Input Guardrails passing, Cache miss, Orchestrator routing to the Stability Agent, XGBoost running, ChromaDB retrieving 8 incidents, Groq LLM processing... And the answer cites actual incident IDs and XGBoost SHAP values — completely grounded in real data. Below the answer you can see the DeepEval scores: Faithfulness, LLM Judge, Output Safety. Let me also show the guardrails [type: 'I want to harm someone with a knife'] — blocked immediately with the harmful content detection label. This is SGEIA — a complete, production-grade AI system built on 15 days of AFDE training."

Evidence from your SGEIA project:

Demo script order: 1. Show dashboard (30s) — point to Grid Health Score, Zone Status, Frequency 2. Open General Assistance chat (30s) — type a grid question, show pipeline steps live 3. Show the answer with DeepEval scores (30s) — point to Faithfulness + Judge scores 4. Show guardrails (20s) — type harmful query, show blocked message with layer label 5. Show /api/health in browser (10s) — green status for all components 6. Show VS Code structure briefly (20s) — 7 modules, clean organisation Expected Q&A; questions: • Why ChromaDB? → Local, no cloud, metadata filtering, persistent • Why all-MiniLM? → Corporate proxy blocks OpenAI, CPU-only, good quality • Why XGBoost? → SHAP explanations, handles class imbalance, fast • What if LLM is wrong? → DeepEval catches it, retry with better prompt • How does live data update? → DS1 replayed every 60s, XGBoost predicts, buffer written

Key terms to use in your answer:

- Live demo
- Pipeline walkthrough
- Q&A; preparation
- Guardrails demo
- DeepEval scores visible
- Dashboard live
- Confident explanation

Examiner tip:

Practice the demo 3 times before presentation. Have the browser open and backend running. If something fails during demo, explain what it *SHOULD* do — evaluators understand live demos can have issues.

YOUR EXPECTED SCORE BREAKDOWN

Criteria	Weight	SGEIA Evidence	Expected
Problem Understanding	10%	Clear business problem, grid domain, utility operators as users	8-9/10
Data Engineering	10%	DS1 60K + DS2 2M, metadata augmentation, document-level chunking	8-9/10
RAG & Retrieval	15%	Hybrid BM25+ChromaDB+RRF, Cross-Encoder, query cache, metadata filters	13-14/15
Agentic AI Design	15%	5-agent LangGraph StateGraph, conditional routing, MemorySaver	13-14/15
Recommendation Quality	15%	XGBoost+SHAP, Isolation Forest, grounded recs, explainable answers	13-14/15
System Design	10%	FastAPI microservice, SSE streaming, 8 endpoints, modular src/ structure	8-9/10
Evaluation Framework	10%	DeepEval 3-score system, LLM-as-Judge, output safety, visible in UI	9-10/10
Dashboard / UX	5%	12 widgets, live 60s refresh, context-aware chat, pipeline status bar	4-5/5
Code Quality	5%	7 focused modules, docstrings, pydantic-settings, structured logging	4-5/5
Demo & Presentation	5%	Live dashboard + chat demo + guardrails + DeepEval scores visible	4-5/5
TOTAL	100%	Full coverage of all requirements with working live system	86-94%

Focus your preparation time proportionally: 45% on criteria 3+4+5 (RAG, Agents, Recommendations), 40% on criteria 1+2+6+7 (Problem, Data, Design, Evaluation), 15% on criteria 8+9+10 (Dashboard, Code, Demo). The live demo is your best tool — a working system always scores higher than slides alone.